

User-Space I/O Software

1 Basic Idea

Definition

User-space I/O software is the part of the I/O system that runs outside the kernel, mainly as library procedures linked with user programs or as user-level service programs.

- Most I/O software is inside the operating-system kernel.
- A smaller part runs in user space.
- User-space I/O includes library routines such as `write`, `printf`, and `scanf`.
- It can also include user-level programs such as printer daemons and network daemons.
- User-space I/O sits at the top of the I/O software stack.

Core Point

User-space I/O software prepares, formats, and queues I/O requests before the kernel and device layers perform the actual hardware work.

2 Library Procedures for System Calls

System Call Wrapper

Many I/O system calls are invoked through library procedures that package parameters and enter the kernel.

```
count = write(fd, buffer, nbytes);
```

The procedure `write` may be linked into the program binary, or it may be loaded dynamically during execution. The wrapper places parameters where the system-call mechanism expects them and then executes the system-call entry instruction.

3 Formatting I/O

Formatting Work

Some user-space I/O procedures do real work before calling the kernel, especially input and output formatting.

- `printf` takes a format string and values.
- It builds an ASCII character string.

- It then calls `write` to output the resulting string.
- `scanf` reads input and stores parsed values into variables described by a format string.

Why Formatting Is User-Level

Formatting does not require direct hardware access, so it can safely be done by user-space library code.

4 Printf Example

Example Statement

```
printf("The square of %3d is %6d\n", i, i*i);
```

- The literal text `The square of` is copied into the output string.
- The value of `i` is formatted as a 3-character decimal field.
- The literal text `is` is added.
- The value `i2` is formatted as a 6-character decimal field.
- A newline character is appended.
- The completed string is passed to `write`.

Important Detail

`printf` does not print directly to the hardware. It formats data, then uses lower I/O layers through a system call.

5 Spooling

Definition

Spooling is a user-level technique for managing dedicated I/O devices by placing work in a queue directory and letting a daemon process perform the actual device access.

Spooling is useful for dedicated devices such as printers. Direct printer access is risky because a process could open the printer and keep it unavailable for hours. Instead, users place complete print jobs in a spooling directory, and a daemon is the only process allowed to open the printer's special file.

6 Printer Spooling Flow

1. User process generates the complete file to print.
2. User process places the file in the spooling directory.

3. The printer daemon notices or selects the queued file.
4. The daemon opens the printer special file.
5. The daemon sends the file to the printer.
6. When done, the daemon moves to the next queued job.

Spooling Benefit

Spooling turns a dedicated device into a queued service, so many users can submit work without directly owning the device.

7 Other Uses of Spooling

Spooling is not only for printers. Network file transfer can use a network daemon and a network spooling directory. News systems historically used spooling to queue messages for later distribution. These systems can run outside the operating-system kernel.

8 Complete I/O Software Stack

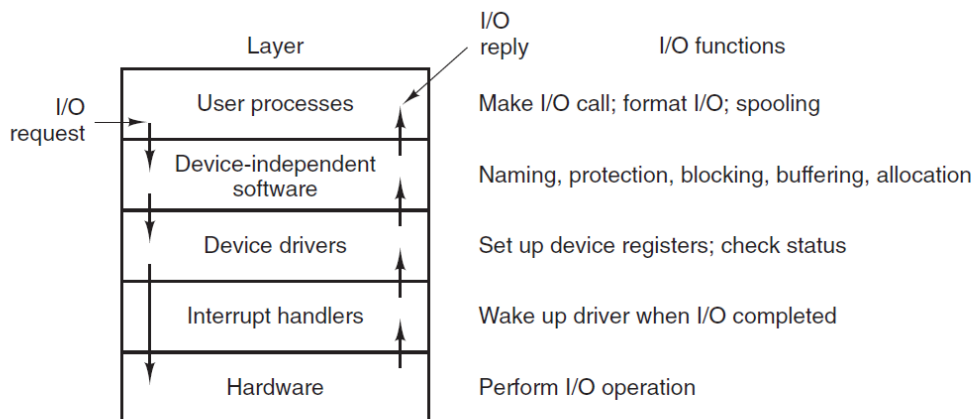


Figure 5-17. Layers of the I/O system and the main functions of each layer.

9 Control Flow for a Read

1. A user program requests a block from a file.
2. The kernel is entered through an I/O system call.
3. Device-independent software checks whether the block is already in the buffer cache.
4. If the block is not cached, the device driver is called.
5. The driver issues the hardware request to the disk controller.
6. The requesting process blocks.

7. When the disk finishes, hardware raises an interrupt.
8. The interrupt handler identifies the device and extracts status.
9. The sleeping process or driver is awakened.
10. The I/O request is finished and the user process continues.

10 Layer Responsibilities

Layer	Responsibility
User processes	Make I/O calls, perform formatted I/O, and use spooling services.
Device-independent OS software	Handles naming, protection, blocking, buffering, and allocation.
Device drivers	Set up controller registers and check device status.
Interrupt handlers	Wake up drivers or processes when I/O completes.
Hardware	Performs the actual I/O operation.

Final Point

User-space I/O software is the top layer of the I/O system: it formats requests, invokes system calls, and can manage queued I/O services such as spooling.